Contents

1. Moodle Quiz Besprechung
2. Häufige Fehler aus Exercise Sheet 4.
3. In-Class Aufgaben
   - 3.1. Max-Heap
   - 3.2. Binary Trees
   - 3.3. 2-3 Trees
   - 3.4. Dynamic Programming

1. Moodle Quiz Besprechung

Aufgabe 1.

Which recurrence describes the running time of Mergesort for $n \geq 1$, assuming n is a power of two?

- a. $T(n) = 2T(n/2) + \Theta(n^2)$
- b. $T(n) = 2T(n/2) + \Theta(n)$
- c. $T(n) = 2T(n-1) + \Theta(n^2)$
- d. $T(n) = 2T(n/2) + \Theta(1)$

Lösung:

$T(n)$ ist die Laufzeit von Mergesort$(A[1,\dots,n], 1, n)$

```
Mergesort (A[1 ... n], l, r)
  if l < r then
    m ← ⌊(l+r)/2⌋
    Mergesort (A, l, m)   → Laufzeit T(n/2)
    Mergesort (A, m+1, r) → Laufzeit T(n/2)
    Merge (A, l, m, r)    → Laufzeit Θ(n)
```

Insgesamt $T(n) = 2T(\frac{n}{2}) + \Theta(n)$

$\Rightarrow T(n) = 2T(\frac{n}{2}) + \Theta(n) \Rightarrow$ b.)

Aufgabe 2

Consider the Quicksort algorithm. Given the array $[7, 2, 9, 4, 6]$, the last element is chosen as the pivot. At which position in the array is the pivot after completely partitioning all elements? Positions are numbered starting at 1.

Answer:

Lösung:
6 ist Pivot
Es gibt 2 Elemente kleiner als 6 → 2,4
Es gibt 2 Elemente größer als 6 → 7,9
=> 6 hat Index 3

Aufgabe 3

Quicksort can partition an array *in place*, using only $O(1)$ additional storage.

- True
- False

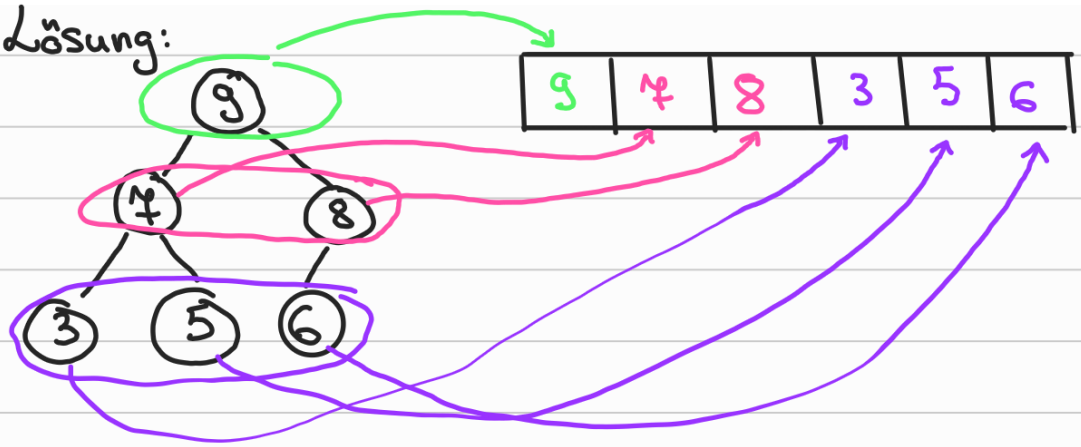
Lösung:
Die Partitionierung ordnet die Elemente innerhalb des ursprünglichen Arrays neu an und verwendet für die Vertauschungen nur eine konstante Anzahl an Hilfsvariablen.

Aufgabe 4

In the lecture you have seen that a complete binary tree can be stored as an array, where nodes are stored level by level, from left to right, starting at index 1 of the array.
Which of the following array representations of a complete binary tree corresponds to a valid Max-Heap? Choose exactly one answer.

- a. $[7, 9, 8, 3, 5, 6]$
- b. $[9, 7, 8, 3, 5, 6]$
- c. $[9, 7, 6, 3, 5, 8]$
- d. $[9, 7, 8, 3, 10, 6]$

Lösung:



=> b) [9,7,8,3,5,6]

Aufgabe 5

Why must the decision tree of a correct comparison-based sorting algorithm for n distinct elements contain at least $n!$ leaves?

- a. Because every path in the decision tree contains exactly $n!$ comparisons.
- b. Because each comparison has n possible outcomes.
- c. Because $n!$ different orderings of the input elements must be distinguished.
- d. Because every element must be compared with every other element.

Lösung: c)

Aufgabe 6

A Max-Heap contains the key 15 at its root. Where would you continue searching for the key 8?

- a. In the left subtree.
- b. In the right subtree.
- c. We cannot know which of the two subtrees might contain the key.

Lösung:
Bei einem Max-Heap besteht die einzige Beziehung zwischen den Knoten darin, dass jeder Elternknoten grösser ist als seine Kinder. Diese Definition liefert uns keine weitere Informationen über die Position der Schlüssel, die kleiner als die Wurzel sind.
=> c.)

Aufgabe 7

A binary search tree contains the key 15 at its root. Where would you continue searching for the key 8?

- a. In neither subtree, because we already know that the key is not contained in the binary search tree.
- b. In the right subtree.
- c. In the left subtree.

Lösung: Die Eigenschaft eines binären Suchbaums besteht darin, dass die Schlüssel, die kleiner als der Schlüssel eines gegebenen Knotens sind, in seinem linken Teilbaum liegen und die grösseren in seinem rechten Teilbaum.
=> c.)

Aufgabe 8

A 2-3 tree of height h has at most 3^h leaves.

- True
- False

Lösung: Jeder Knoten, der kein Blatt ist, hat höchstens 3 Kinder. Das bedeutet, dass es auf Ebene h' höchstens $3^{h'}$ Knoten gibt, und wenn der Baum Höhe h hat, dann hat er höchstens 3^h Blätter.

Aufgabe 9

A singly linked list L already stores a pointer to its first element. Assume that an additional pointer to its last element is now stored. Which operation improves from $\Theta(n)$ to $\Theta(1)$ in the worst case, where n denotes the length of the list?

a. Searching for a given key value.

b. Appending a new key k at the end using $\texttt{insert}(k, L)$.

c. Obtaining the memory address of the i-th element using $\text{get}(i, L)$, where $i \in \{1, \ldots, n\}$ is arbitrary.

d. Deleting an arbitrary node whose memory address is known.

Lösung: Die einzige zusätzliche Information, die wir erhalten, ist ein Pointer auf das letzte Element. Das bedeutet, dass die verbesserte Operation direkt davon profitiert, dass wir den Zeiger auf das letzte Element direkt haben, anstatt die Liste durchlaufen zu müssen, um ihn zu erhalten. Die einzige der aufgeführten Operationen, die nur auf den Zeiger auf das letzte Element angewiesen ist, ist das Einfügen eines Elements in die Liste.
=> B.)

Aufgabe 10

Consider the abstract datatype (ADT) *List*. The memory address of the node to be deleted is given. The address of its predecessor is **not** provided separately. Which statement describes the worst-case running time for the deletion of the node in a list with n elements?

- a. Singly linked lists require $\Theta(n)$, while doubly linked lists require $\Theta(1)$.
- b. Both singly and doubly linked lists require $\Theta(n)$.
- c. Both singly and doubly linked lists require $\Theta(1)$.
- d. Singly linked lists require $\Theta(1)$, while doubly linked lists require $\Theta(n)$.

Lösung: a)

Aufgabe 11

What is the running time of the following program?

```
FIBONACCI(n)
1 F[1..n] ← new array
2 F[1] ← 1
3 F[2] ← 1
4 for i ← 3, . . . , n do
5     F[i] ← F[i − 1] + F[i − 2]
6 return F[n]
```

- a. $\Theta(1)$
- b. $\Theta(n)$
- c. At least exponential
- d. None of the other options

Lösung: Wir zählen die Anzahl der elementaren Operationen, in diesem Fall Additionen.

Base case: $\Theta(1)$ ($F[1] \leftarrow 1$, $F[2] \leftarrow 1$)

Filling in DP Table: $\sum_{i=3}^{n} 1 = n-2 = \Theta(n)$

Extracting the final answer: $\Theta(1)$ (return F[n])

=> B.) $\Theta(n)$

Aufgabe 12

Consider the jump game from the lecture with given input array $A[1..n]$. Let $S[i]$ denote the minimum number of jumps needed to reach reach i. Which of the following expressions is the correct recursion for $S[i]$?

- a. $S[i] = \min\{S[j] + 1 \mid 1 \leq j < i \text{ and } j + A[j] \geq i\}$
- b. $S[i] = \min\{A[j] + 1 \mid 1 \leq j < i \text{ and } j + A[j] \leq i\}$
- c. $S[i] = \min\{A[j] + 1 \mid 1 \leq j < i \text{ and } j + A[j] \geq i\}$
- d. $S[i] = \min\{S[j] + 1 \mid 1 \leq j < i \text{ and } j + A[j] \leq i\}$

Lösung: a)

Aufgabe 13

For the edit distance $ED(n, m)$ between two given strings $A[1..n]$ and $B[1..m]$, assume that after the optimal sequence of edits, the entry $A[n]$ is not deleted and ends up in a position in $B[1..m-1]$. What is then necessarily true?

- a. $ED(n, m) = \begin{cases} 0 & \text{if } A[n] = B[m] \\ 1 & \text{if } A[n] \neq B[m] \end{cases}$
- b. $ED(n, m) = 1 + ED(n-1, m)$
- c. $ED(n, m) = 1 + ED(n-1, m-1)$
- d. $ED(n, m) = 1 + ED(n, m-1)$

Lösung: d.)

2 Häufige Fehler aus Exercise Sheet 4

Exercise 4.4

Exercise 4.3 *One-Looped Sort* **(1 point).**

Consider the following pseudocode whose goal is to sort an array A containing n integers.

Algorithm 1 Input: array $A[0 \dots n-1]$.

```
i ← 0
while i < n do
    if i = 0 or A[i] ≥ A[i − 1] then:
        i ← i + 1
    else
        swap A[i] and A[i − 1]
        i ← i − 1
```

(b) Explain why the algorithm correctly sorts any input array. Formulate a reasonable loop invariant, prove it (e.g., using induction), and then conclude using that invariant that the algorithm correctly sorts the array.

Hint: *Use the invariant "at the moment when the variable i gets incremented to a new value $i = k$ for the first time, the first k elements of the array are sorted in increasing order".*

Common mistakes:

1) Incorrect induction step

2) Missing conclusion

1 Incorrect induction step

We want to show that the invariant from the hint holds for every $1 \leq k \leq n$

Base case: $k = 1$

We know that since we enter the loop with $i=0$, we always enter the if condition on the first iteration of the while loop.

=> i is incremented to 1 for the first time on the first iteration and the subarray $A[0]$ contains 1 element so it is trivially sorted.

=> the invariant holds for $i = k$

Induction Hypothesis: We assume the invariant holds for $k = p$, $1 \leq p \leq n$.

=> The first time i is incremented to p the first p elements of A are sorted in increasing order

Induction Step: We will show the invariant holds for $k = p+1$

We know that $i=p$ is incremented to $i = p+1$ only when $A[i] \geq A[i-1]$. Since $i = p$, from IH we know that $A[0, \dots, p-1]$

is sorted in increasing order and since $A[i] \geq A[i-1]$ and $i = p$ $A[0, \ldots, p]$ is also sorted when i is increased to $p+1$.
=> We have shown that the invariant holds for $i = p+1$.

Using the induction hypothesis in this way is wrong. It is true that the only way to increase i to $p+1$ is when $i=p$ to hold $A[i] \geq A[i-1]$. However, written as it is, we have no way of knowing if when $A[i] \geq A[i-1]$, $i=p$ for the first time. Therefore we cannot use the assumption we made in IH. In order to use this asumption, we need to enter the IS from the perspective that i has just been increased to p for the first ime and from there consider the 2 cases if $A[i] \geq A[i-1]$ or $A[i] < A[i-1]$

2 Missing conclusion

The goal of this part of the exercise is to prove the correctness of the algorithm. As part of the proof you show the correctness of the invariant, but this by itself is not quite enough. You need to use the corectness of the invariant and show how from it you can derive the correctness of the algorithm. You need the conclusion, otherwise the solution is incomplete.
Example conclusion from the master solution.

Exercise 4.5

Algorithm 4

(a)
```
i ← 1
while i ≤ n do
    j ← i
    while 2^j ≤ n do
        f()
        j ← j + 1
    i ← i + 1
```

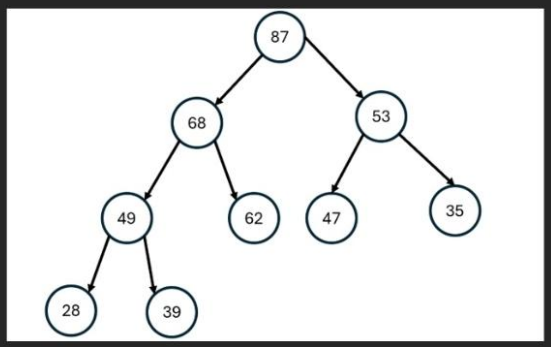
Using $\log n$ instead of $\lfloor \log n \rfloor$

In this exercise you are asked not only about the asymptotic bound for the number of calls of f(), but also about the exact number of calls of f(). Since the loops do integer number of iterations and the function f() can only be called an integer number of times. For that reason using $\log n$ instead of $\lfloor \log n \rfloor$ is not precise enough.

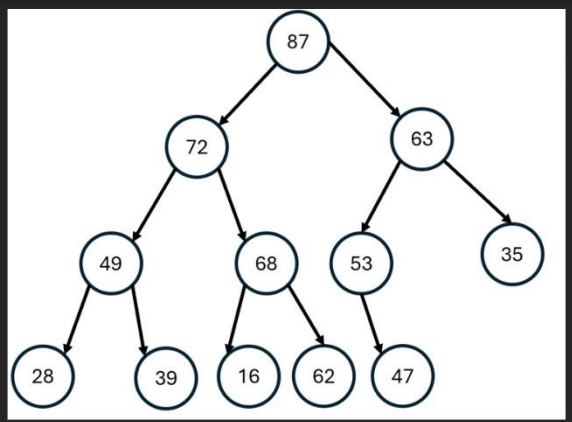
3. In-Class Exercises

3.1. Max-Heap

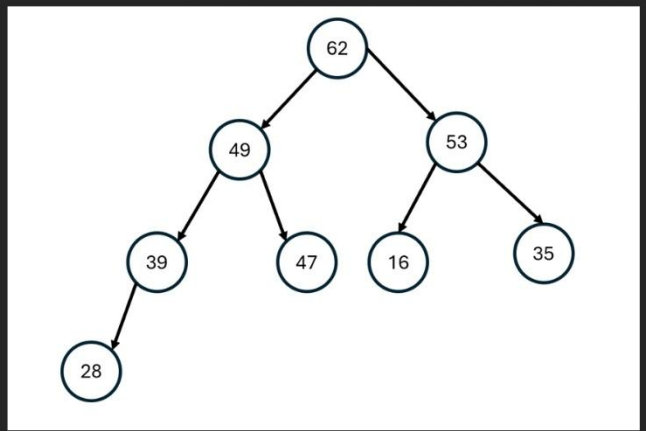
You are given the following Max-Heap.



Now we insert the keys 16, 72, 63 in this order. What does the Max-Heap look like?



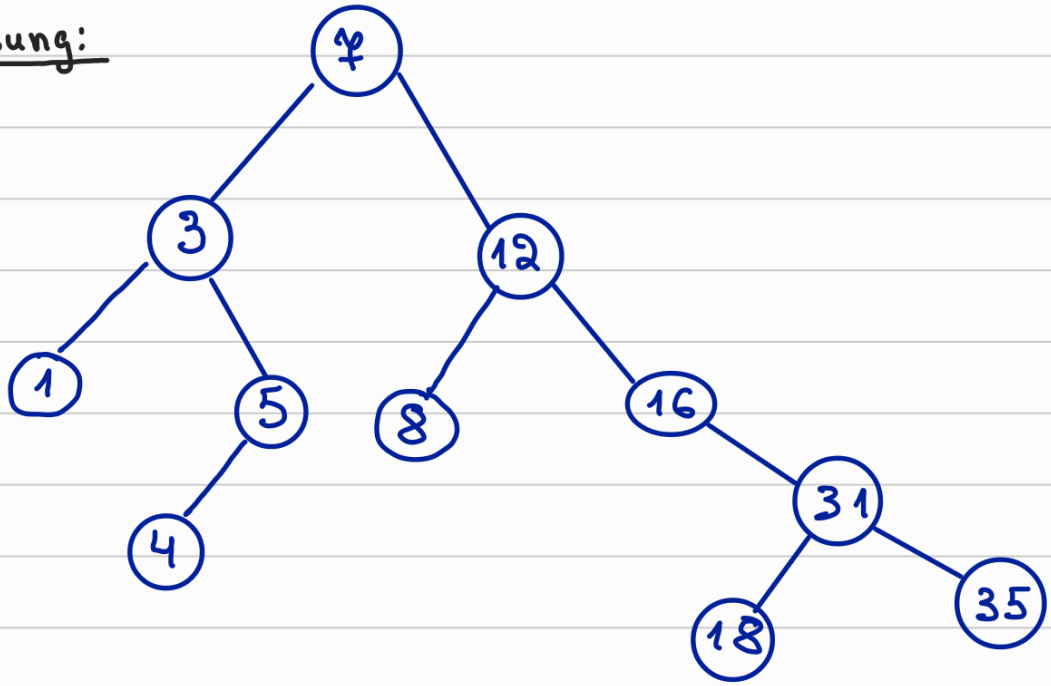
Now we extract the max key 4 times. What does the Max-Heap look like?



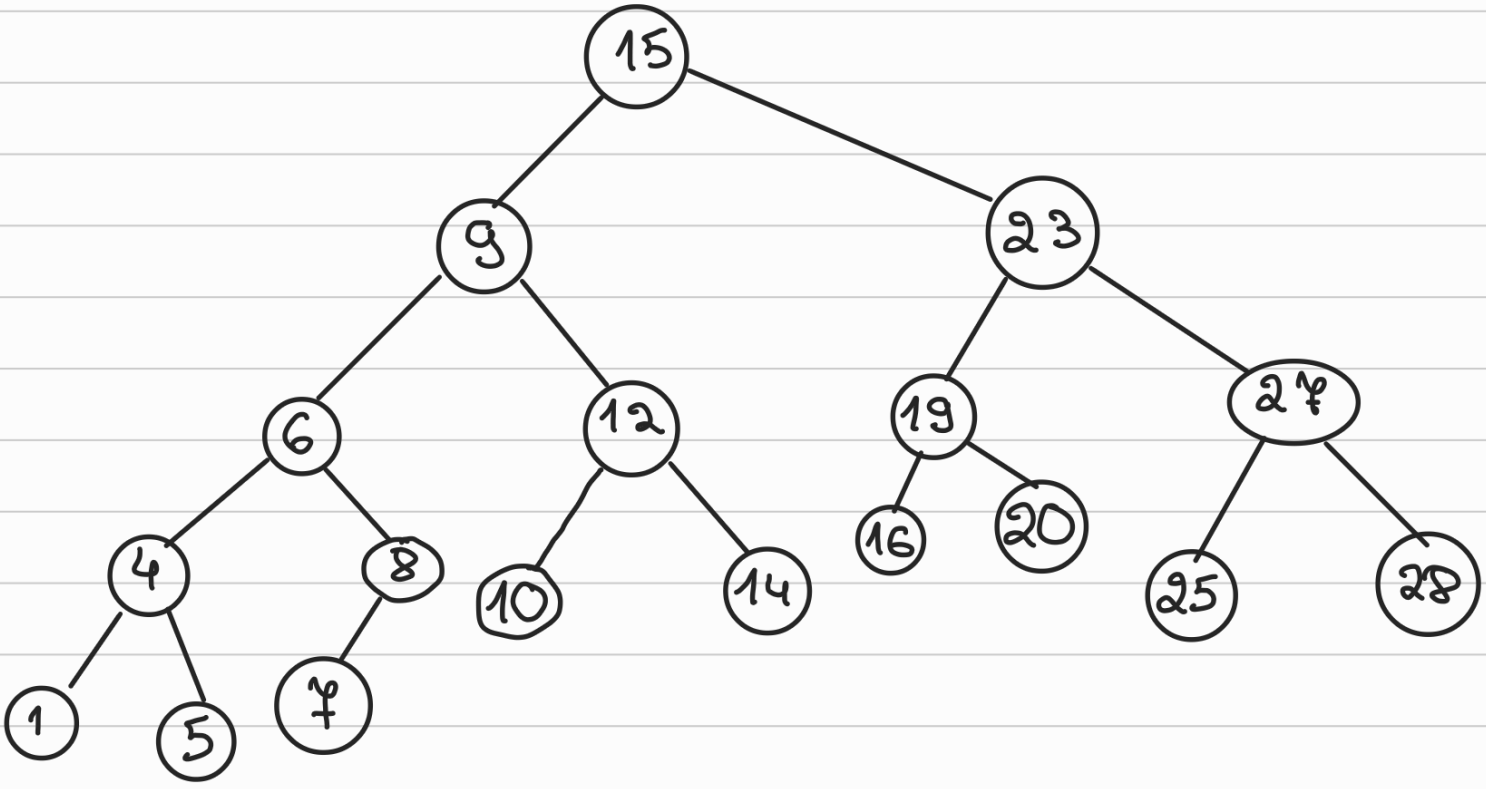
3.2. Binary Search Trees

Exercise 2 The keys 7, 3, 12, 5, 16, 8, 31, 35, 4, 1, 18 are inserted in an empty Binary search tree in that order. Draw the BST.

Lösung:

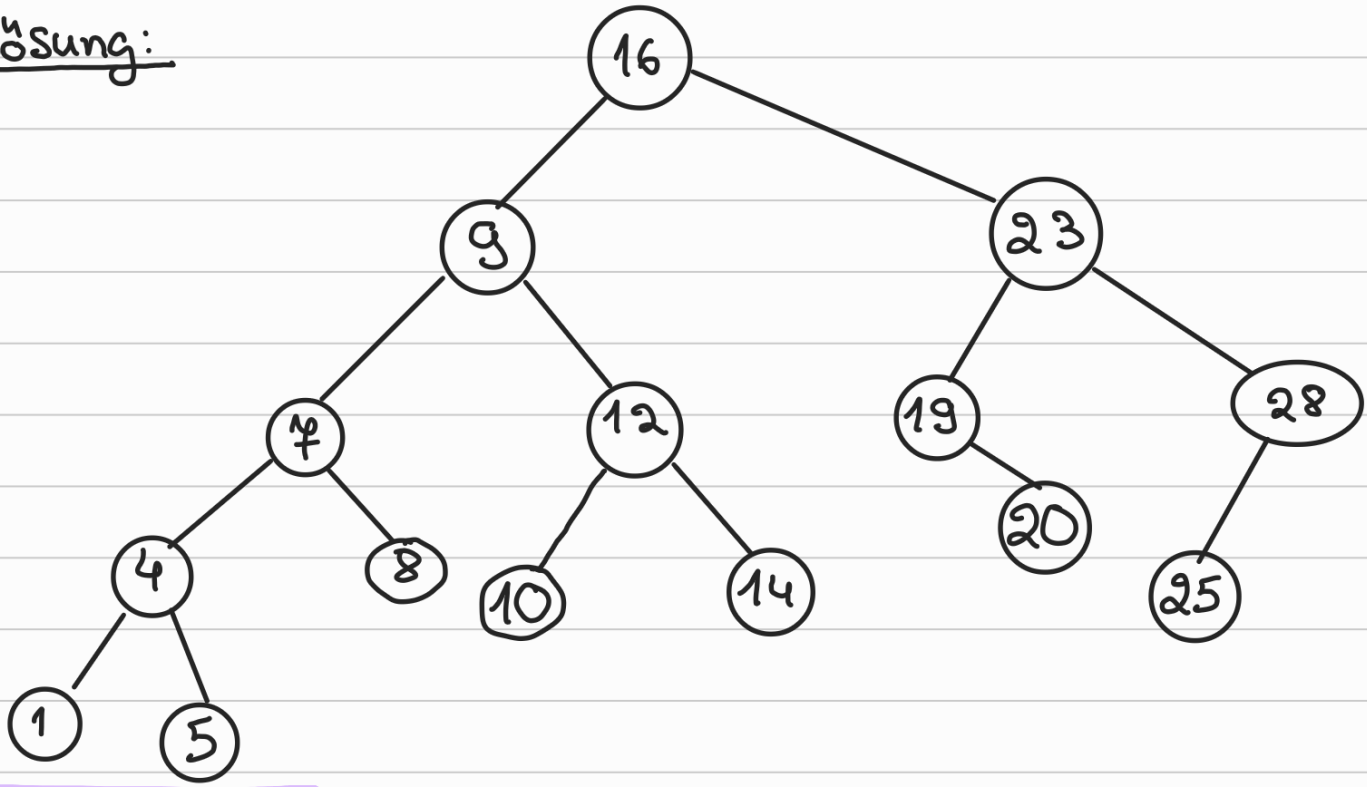


Exercise 3 You are given the following BST.



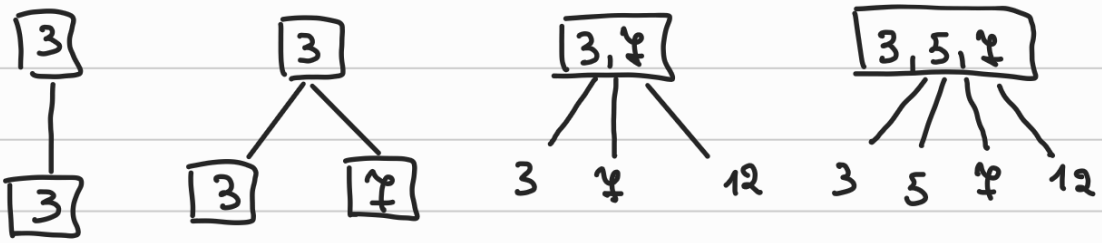
Delete 6, 15, 27 in this order.
Draw the resulting BST.

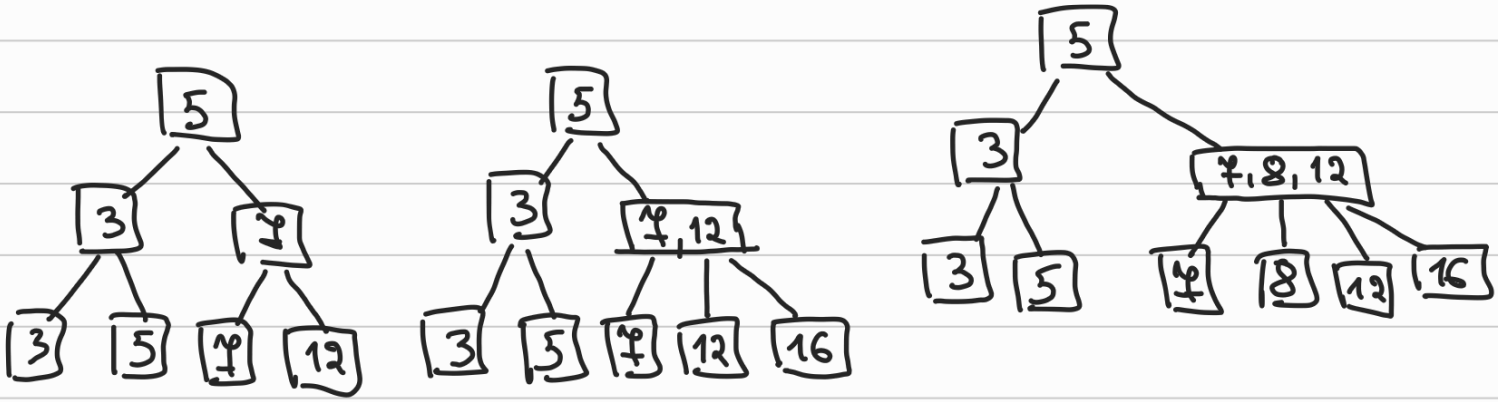
Lösung:

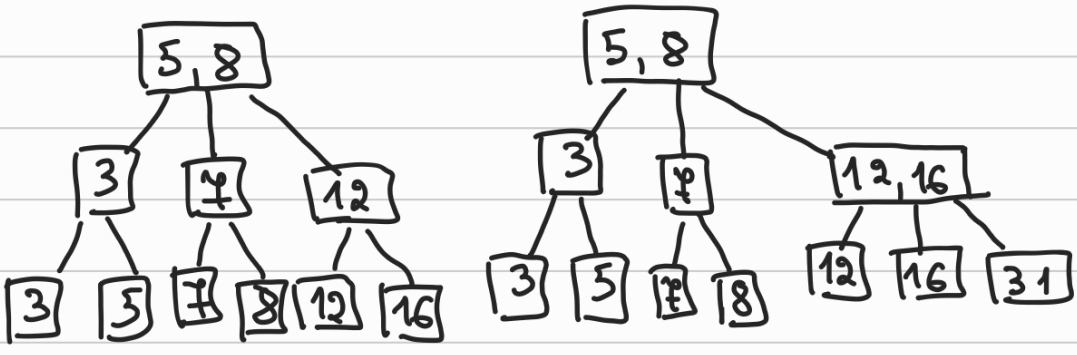


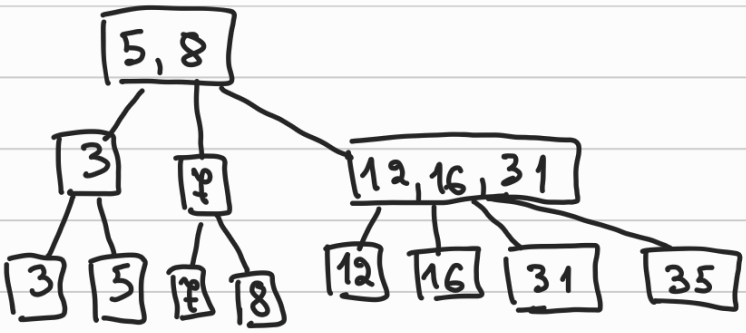
3.3. 2-3 Trees

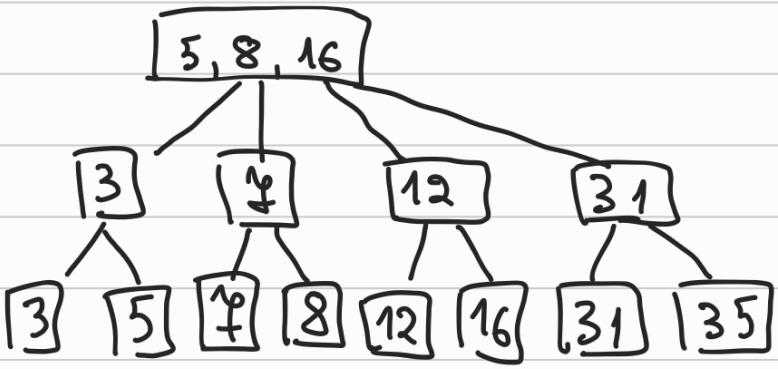
Exercise 4 The keys 3, 7, 12, 5, 16, 8, 31, 35, 33 are inserted in an empty 2-3-Tree in this order. Draw the 2-3 tree.

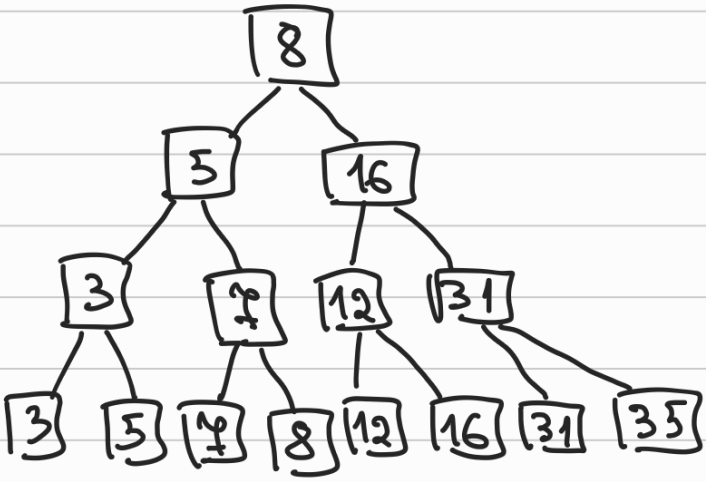


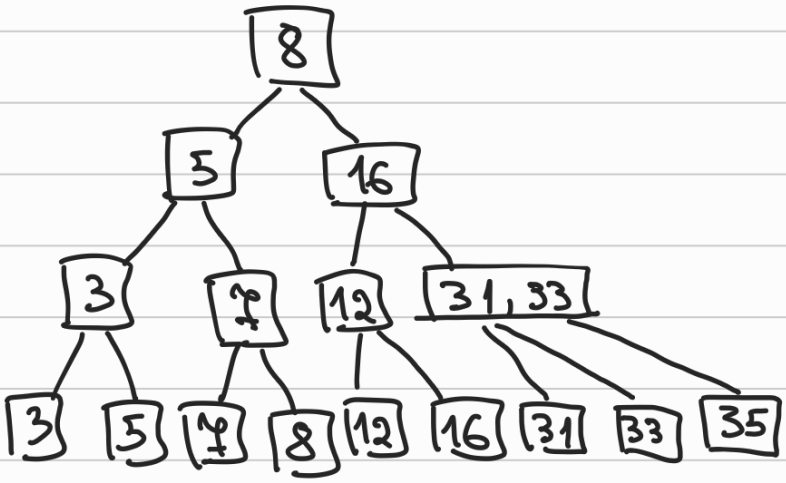




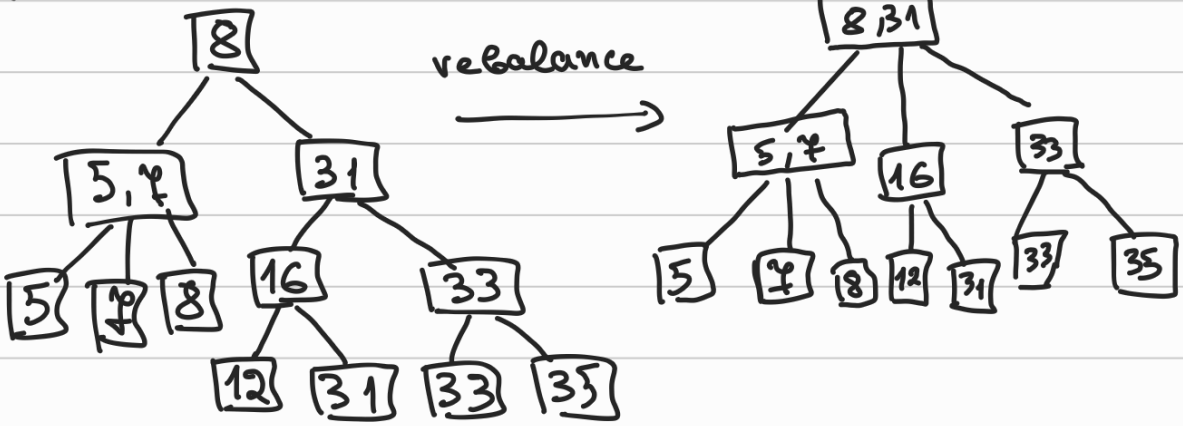








Exercise 4. From the above tree delete 16, 5 in this order



3.4 Dynamic Programming

Exercise 5 Sequence

We are given a sequence of n numbers. Some numbers of the sequence we multiply by 2, others by 3. It is forbiden

to multiply 2 consecutive numbers by 3. The numbers are in $\mathbb{Z}$. After we multiply the numbers we sum them. Find an algorithm that returns the biggest possible sum. Argue about its correctness and runtime.

Exercise 6 Sequences

We are given an integer p, $p \in \mathbb{N}$. Find an algorithm which computes the number of different sequences of n nonnegative integers, that are all less than p and have sum less than a given integer s. Argue about its correctness and runtime.

Exercise 7 Jumps

A frog is in the start $(0,0)$ of a coordinate systhem. It wants to reach point (N,M) in exactly K seconds. In every second the frog can increase its x coordinate by at most A and its y coordinate by at most B (it can increase both its coordinates independently in the same second. Find an algorithm that computes in how many different ways the frog can reach